

MODUL 12. Laravel : Database 1

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi CodeIgniter pada pembuatan aplikasi berbasis *web*.
2. Mahasiswa mampu menerapkan CRUD menggunakan Laravel dan konsep MVC.
3. Mahasiswa mampu mengimplementasikan fitur-fitur yang sering digunakan pada Laravel.

12.1 CRUD

Pada modul ini kita akan membuat aplikasi yang dapat melakukan create, read, update, dan delete terhadap suatu data/tabel. Aplikasi yang akan kita bangun yaitu aplikasi e-commerce dimana kita akan fokus melakukan CRUD terhadap tabel produknya saja.

12.1.1 Konfigurasi dan Skema

Hal pertama yang harus dilakukan agar aplikasi dapat terhubung dengan database adalah mengatur konfigurasi koneksi. Secara detail, konfigurasi database berada pada file **config/database.php**. Tetapi untuk konfigurasi standar, kita dapat mengubahnya di file **.env** pada baris yang berisi **DB_CONNECTION** hingga **DB_PASSWORD**. Ubah nilainya dengan pengaturan yang kita inginkan.

```
...
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=
DB_CHARSET=utf8mb4
DB_COLLATION=utf8mb4_unicode_
ci
...
```

Buat database bernama **ecommerce** di DBMS. Untuk membuat tabelnya, dapat dilakukan secara otomatis (di-generate) oleh Laravel dari command prompt / console / terminal dengan perintah. Perintah yang pertama yaitu membuat file migrasi sebagai skema dari tabel:

```
php artisan make:migration create_products_table
```

File migrasi **xxx_create_products_table.php** akan terbuat pada folder **database/migrations**. Edit file ini pada fungsi **Schema::create()** untuk mendefinisikan kolom yang kita inginkan.

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->integer('price');
    $table->timestamps();
});
```

Penjabaran dari kode ini:

- Fungsi **id()** yaitu kita mendefinisikan kolom **id** sebagai PK dengan tipe int dan auto-increment.
- Fungsi **string('name')** yaitu kita mendefinisikan kolom **name** dengan tipe varchar
- Fungsi **integer('price')** yaitu kita mendefinisikan kolom **price** dengan tipe int
- Fungsi **timestamps()** yaitu kita mendefinisikan kolom **created_at** yang akan terisi jika data

dibuat melalui ORM dan **updated_at** yang akan terisi jika data diubah melalui ORM

Untuk membuat tabel ke database dari skema ini, maka ketikkan perintah berikut pada command prompt / console / terminal:

```
php artisan migrate
```

12.1.2 Model

Laravel memberikan tiga cara untuk mengakses ataupun manipulasi data ke database: query langsung, query builder, dan ORM. Query langsung dan query builder menggunakan library Illuminate (**Illuminate\Support\Facades\DB**) sedangkan ORM menggunakan Eloquent, yang merupakan kelas extend dari Illuminate. File Model diletakkan pada folder **app/Models**. Untuk membuat Model dalam Laravel, dapat dilakukan dengan manual dengan menambahkan namespace "App\Models" dan meng-extend kelas base Model dari Eloquent (**Illuminate\Database\Eloquent\Model**) ataupun di-generate oleh Laravel dari command prompt / console / terminal dengan perintah:

```
php artisan make:model Product
```

Jika sebelumnya belum membuat Controller atau file migration-nya, perintah generate Model ini dapat ditambahkan parameter sehingga bisa sekaligus me-generate file Controller-nya (-c), file migration-nya (-m), ataupun keduanya langsung (-cm).

Eloquent memiliki beberapa konvensi / aturan yang harus diperhatikan yaitu:

- Nama sebuah Model "X" secara otomatis merepresentasikan tabel database bernama "xs". Contoh, Model Product akan merepresentasikan tabel **products**. Jika tabel yang direpresentasikan berbeda, maka harus ditambahkan atribut **protected \$table = 'nama_tabel'**; pada kelas Model.
- Kolom PK pada tabel bernama "id". Jika kolom PK bukan "id", maka harus ditambahkan atribut **protected \$primaryKey = 'nama_kolom_PK'**; pada kelas Model.
- Kolom PK pada tabel di-set auto-increment. Jika kolom PK tidak auto-increment, maka harus ditambahkan atribut **public \$incrementing = false**; pada kelas Model.
- Kolom PK pada tabel bertipe integer. Jika kolom PK bukan integer, maka harus ditambahkan atribut **protected \$keyType = 'string'**; pada kelas Model.
- Ada kolom "created_at" dan "updated_at" pada tabel. Jika tidak ada, maka harus ditambahkan atribut **public \$timestamps = false**; pada kelas Model.

Beberapa aturan lain juga dapat diatur seperti date format, koneksi DB yang berbeda, dan nilai default untuk atribut tertentu.

12.1.3 Controller

Setelah Model siap, kita tinggal memanggilnya pada Controller. Berikut kode lengkapnya dalam ProductController:

```
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
```

```

{
    $products = Product::all();
    return view('products.index', ['products' => $products]);
}

public function create()
{
    return view('products.form', [
        'title' => 'Tambah',
        'product' => new Product(),
        'route' => route('products.store'),
        'method' => 'POST',
    ]);
}

public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    Product::create($validated);
    return redirect()->route('products.index')->with('success', 'Produk berhasil
ditambahkan');
}

public function show(Product $product)
{
    return view('products.show', compact('product'));
}

public function edit(Product $product)
{
    return view('products.form', [
        'title' => 'Edit',
        'product' => $product,
        'route' => route('products.update', $product),
        'method' => 'PUT',
    ]);
}

public function update(Request $request, Product $product)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);

    $product->update($validated);
    return redirect()->route('products.index')->with('success', 'Produk berhasil
diperbarui');
}

public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index')->with('success', 'Produk berhasil
dihapus');
}
}

```

12.1.4 View

Untuk tampilannya, buat file dengan nama **index.blade.php** yang diletakkan di folder **resources/views/product** (folder product dibuat dahulu agar rapi).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Daftar Produk</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
    <div class="row justify-content-center mt-5">
        <div class="col-md-8">
            @if(session('success'))
                <div class="alert alert-success">{{ session('success') }}</div>
            @endif
            <div class="d-flex justify-content-between align-items-center mb-3">
                <span>{{ session('msg') }}</span>
                <a href="{{ route('products.create') }}" class="btn btn-
primary">Tambah</a>
            </div>
            <table class="table table-bordered table-striped">
                <thead>
                    <tr>
                        <th>Nama</th>
                        <th>Harga</th>
                        <th>Aksi</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach($products as $product)
                        <tr>
                            <td>{{ $product->name }}</td>
                            <td>{{ $product->price }}</td>
                            <td>
                                <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-primary">Edit</a>
                                <form method="POST" action="{{ route('products.destroy',
$product->id) }}" style="display:inline" onsubmit="return confirm('Yakin hapus?')">
                                    @csrf
                                    @method('DELETE')
                                    <button class="btn btn-sm btn-danger">Hapus</button>
                                </form>
                            </td>
                        </tr>
                    @endforeach
                </tbody>
            </table>
        </div>
    </div>
    <script
src="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></
script>
</body>
</html>
```

Dan untuk tampilan form tambah dan edit, dapat dibuat satu halaman saja karena hampir sama. Hanya perlu tambahan pengkondisian di beberapa tempat.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form {{ $title }} Produk</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
  <div class="row justify-content-center mt-5">
    <div class="col-md-6">
      <h4>Form {{ $title }} Produk</h4>
      <form class="border p-4" method="POST" action="{{ $route }}">
        @csrf
        @if($method === 'PUT')
          @method('PUT')
        @endif
        <div class="mb-3">
          <label for="name" class="form-label">Nama</label>
          <input type="text" name="name" id="name" class="form-control
@error('name') is-invalid @enderror" value="{{ old('name', $product->name) }}">
          @error('name')
            <div class="invalid-feedback">{{ $message }}</div>
          @enderror
        </div>
        <div class="mb-3">
          <label for="price" class="form-label">Harga</label>
          <input type="number" name="price" id="price" class="form-
control @error('price') is-invalid @enderror" value="{{ old('price', $product-
>price) }}">
          @error('price')
            <div class="invalid-feedback">{{ $message }}</div>
          @enderror
        </div>
        <div class="text-center">
          <button type="submit" class="btn btn-
success">Simpan</button>
        </div>
      </form>
    </div>
  </div>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.j
s"></script>
</body>
</html>
```

12.1.5 Tampilan Halaman

Jalankan aplikasi. Jika *kode* yang dibuat sesuai dengan semua gambar di atas pada modul ini, maka tampilan aplikasi *webnya* akan seperti gambar dibawah ini.

1. <http://localhost:8000/products>

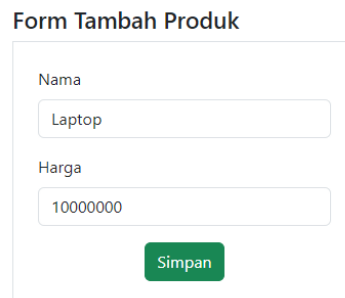


Nama	Harga	Aksi
------	-------	------

Tambah

Gambar 12.1 Tampilan halaman view

2. <http://localhost:8000/products/create>



Form Tambah Produk

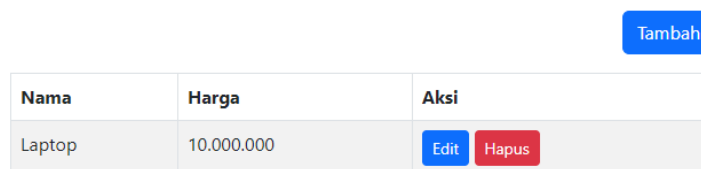
Nama
Laptop

Harga
10000000

Simpan

Gambar 12.2 Tampilan halaman form tambah produk

3. <http://localhost:8000/products>

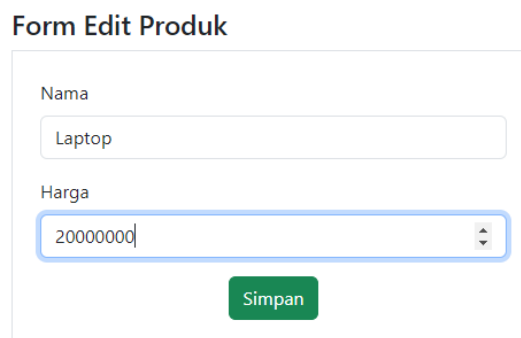


Nama	Harga	Aksi
Laptop	10.000.000	Edit Hapus

Tambah

Gambar 12.3 Tampilan halaman view setelah tambah data

4. [http://localhost:8000/products/\[id\]/edit](http://localhost:8000/products/[id]/edit)



Form Edit Produk

Nama
Laptop

Harga
20000000

Simpan

Gambar 12.4 Tampilan halaman form ubah data

12.2 Templating Halaman

Sebelumnya kita sudah menggunakan beberapa directive Blade. Directive ini juga dapat digunakan untuk templating halaman, yaitu bagian-bagian dari halaman yang sama / berulang untuk semua halaman dapat dibuat menjadi satu file Blade, dan semua halaman itu dapat me-load-nya tanpa harus ditulis ulang.

Langkah pertama dalam templating adalah membuat file template-nya, yang dimiliki oleh tiap halaman. Untuk contoh kasus pada modul ini, kita membuat file template dengan nama **template.blade.php** dengan isi:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>@yield('title')</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <script src="{{ asset('js/jquery.min.js') }}"></script>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
"></script>
</head>
<body style="width:95%">
  @yield('content')
</body>
</html>
```

Directive **@yield** digunakan untuk menandakan bahwa di bagian itulah yang akan berbeda untuk tiap halamannya. Langkah selanjutnya adalah mengubah tampilan di tiap halaman seperti contoh **index.blade.php** di bawah ini:

```
@extends('template')

@section('title', 'Daftar Produk')

@section('content')
<body class="container">
  <div class="row justify-content-center mt-5">
    <div class="col-md-8">
      @if(session('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
      @endif
      <div class="d-flex justify-content-between align-items-center mb-3">
        <span>{{ session('msg') }}</span>
        <a href="{{ route('products.create') }}" class="btn btn-
primary">Tambah</a>
      </div>
      <table class="table table-bordered table-striped">
        <thead>
```



```

        <tr>
            <th>Nama</th>
            <th>Harga</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody>
        @foreach($products as $product)
            <tr>
                <td>{{ $product->name }}</td>
                <td>{{ $product->price }}</td>
                <td>
                    <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-primary">Edit</a>
                    <form method="POST" action="{{ route('products.destroy',
$product->id) }}" style="display:inline" onsubmit="return confirm('Yakin
hapus?') ">
                        @csrf
                        @method('DELETE')
                        <button class="btn btn-sm btn-danger">Hapus</button>
                    </form>
                </td>
            </tr>
        @endforeach
    </tbody>
</table>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js
"></script>
</body>
@endsection

```

Directive **@extend** digunakan untuk menentukan file template mana yang digunakan oleh halaman ini. Sedangkan directive **@section** digunakan untuk mengisi halaman sesuai dengan nama **yield** yang di-define dalam file template. Directive **@section** dapat diisi dengan string ataupun tag HTML.

12.3 Form Validation

Laravel dapat ditambahkan form validation dari sisi server. Di controller, sebelum kita memanggil fungsi untuk menambahkan (fungsi **store**) / mengubah (fungsi **update**), kita dapat menambahkan kode:

```

...
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);
    ...
public function update(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|min:4',
        'price' => 'required|integer|min:1000000',
    ]);
    ...

```

Di kode validasi ini, kita mengatur nama tidak boleh kosong dan minimal 4 karakter, sedangkan harga tidak boleh kosong, harus integer, dan nilai minimal 1.000.000. Kemudian di halaman **form.blade.php** kita harus menambahkan directive **@error** seperti ini:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Form {{ $title }} Produk</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container">
    <div class="row justify-content-center mt-5">
        <div class="col-md-6">
            <h4>Form {{ $title }} Produk</h4>
            <form class="border p-4" method="POST" action="{{ $route }}">
                @csrf
                @if($method === 'PUT')
                    @method('PUT')
                @endif
                <div class="mb-3">
                    <label for="name" class="form-label">Nama</label>
                    <input type="text" name="name" id="name" class="form-control
@error('name') is-invalid @enderror" value="{{ old('name', $product->name) }}">
                    @error('name')
                        <div class="invalid-feedback">{{ $message }}</div>
                    @enderror
                </div>
                <div class="mb-3">
                    <label for="price" class="form-label">Harga</label>
                    <input type="number" name="price" id="price" class="form-
control @error('price') is-invalid @enderror" value="{{ old('price', $product-
>price) }}">
                    @error('price')

```

```

        <div class="invalid-feedback">{{ $message }}</div>
        @enderror
    </div>
    <div class="text-center">
        <button type="submit" class="btn btn-success">Simpan</button>
    </div>
</form>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
</body>
</html>

```

Directive **@error** diletakkan di atribut class pada tag <input> untuk validasinya dan diletakkan di bawah tag <input> untuk menampilkan pesan validasinya. Kemudian agar data pada form di-repopulate, maka isikan atribut value pada tag <input> dengan **old('value-di-atribut-name')**.